



Stockvio — Desktop UI/UX Redesign Plan

An API-driven, role-aware redesign of the Qt desktop client, aligned with the Stockvio web app: navigation, page-by-page layout, component library, user flows, and UX improvements.

Desktop · Qt / C++

Aligned with · Next.js web

Role-aware · RBAC

Repository: felipeact/InventoryQtApp

Generated: June 24, 2026 · **Version:** 1.0

Table of Contents

Stockvio — Desktop (Qt) UI/UX Redesign Plan	3
1. API analysis	4
1.1 Domains, endpoints, permissions	4
1.2 Roles → capabilities (from role.service.ts)	6
2. How the web app does it (the reference IA)	6
3. Navigation / menu structure (role-aware)	6
4. Page-by-page layout	7
4.1 Dashboard (all roles; DashboardPage / TruckStockDashboardPage)	7
4.2 Products (ItemsPage)	7
4.3 Quick Scan (ScanPage; SCAN_IN/SCAN_OUT)	8
4.4 Assets (AssetsPage)	8
4.5 Fleet (tabbed; TrucksPage + StockTemplatesPage + AssignmentsPage)	8
4.6 My Truck (MyTruckStockPage; VIEW_ASSIGNED_TRUCK_STOCK)	8
4.7 Low-Stock Alerts (LowStockAlertsPage; VIEW_LOW_STOCK_ALERTS)	8
4.8 Receipts (ReceiptsPage; APPROVE_RECEIPTS admin, UPLOAD_RECEIPT tech)	8
4.9 Reports (ReportsPage; VIEW_STOCK, audit needs MANAGE_USERS)	8
4.10 Team (UsersPage; MANAGE_USERS)	8
4.11 Settings (SettingsPage; all)	9
5. Component library (build once, reuse everywhere)	9
6. Key user flows	9
7. UX improvements over today	10
8. Mapping to existing Qt files (what changes)	10
9. Suggested implementation order	10

Stockvio — Desktop (Qt) UI/UX Redesign Plan

Driven by the API surface, aligned with the Stockvio web app. This plan analyzes every backend route, maps it to roles, and specifies a screen-by-screen redesign for the Qt desktop client so it matches the web app's information architecture and polish. It is implementation-ready: each screen lists the endpoints it calls, the data it needs, the components to use, and the role gating.

1. API analysis

1.1 Domains, endpoints, permissions

Domain	Endpoint(s)	Permission	Purpose / data
Auth	POST /auth/login , /register , /refresh , /logout , GET /auth/validate , POST /auth/request-reset , /reset-password , /change-password	public / self	Sign in, token refresh, forced password change. Login now returns user.permissions → drives all UI gating.
Users / Team	GET /users , POST /users/invite , PUT /users/:id , POST /users/:id/reset-password , DELETE /users/:id , POST /users/assign-permission , /remove-permission , PATCH /users/me	MANAGE_USERS	List/invite/edit/remove members; per-user permission grants; edit own profile.
Products	GET /products , /products/:id , /products/low-stock , POST /products , PUT /products/:id , DELETE /products/:id , POST /products/scan-in , /scan-out	VIEW_STOCK (read), ADD_PRODUCT (write), SCAN_IN / SCAN_OUT	Catalog + live quantity; barcode scan in/out; low-stock list.
Assets	GET /assets , /assets/:id , POST /assets , PUT /assets/:id , DELETE /assets/:id	VIEW_ASSET / ADD_ASSET / EDIT_ASSET / DELETE_ASSET	Serialized tools/equipment register.
Trucks	GET /truck-stock/trucks , POST , PUT /:id	VIEW_ALL_TRUCKS (read), MANAGE_TRUCK_STOCK (write)	Fleet vehicles + technician assignment.
Templates	GET /truck-stock/templates , /:id , POST , PUT /:id , DELETE /:id	VIEW_TRUCK_STOCK / MANAGE_TRUCK_STOCK	Standard stock lists per trade, with line items.
Assignments	GET /truck-stock/assignments , POST , PUT /:id , DELETE /:id	MANAGE_TRUCK_STOCK / ASSIGN_TRUCK_STOCK	Bind a template to a truck.
Stock ops	GET /truck-stock/my-stock , /low-stock , /movements , PATCH /items/:id/quantity , POST /transfer-to-truck , /use-item	VIEW_ASSIGNED_TRUCK_STOCK , VIEW_LOW_STOCK_ALERTS , VIEW_TRUCK_STOCK , TRANSFER_STOCK_TO_TRUCK	Technician's truck stock; warehouse → truck transfer; consume item; movement history.
Receipts	POST /truck-stock/receipts/upload , POST /receipts , GET /receipts , POST /:id/items , /:id/reconcile , PATCH /:id/status	UPLOAD_RECEIPT (tech), APPROVE_RECEIPTS (admin)	Upload purchase receipt, reconcile vs template, approve/reject.
Reports	GET /reports/inventory-summary , /assets-summary , /audit-logs , /stock-movements	VIEW_STOCK , MANAGE_USERS (audit)	KPI summaries; audit trail; movement log.
Exports	GET /exports/{products,assets,users}/	VIEW_STOCK , MANAGE_USERS	Download data files.

Domain	Endpoint(s)	Permission	Purpose / data
	{csv,xlsx,pdf} , /exports/ company/json		

Out of scope for the desktop client: `/super-admin/*` and `/leads` are the platform-operator and marketing concerns (handled on the web). The desktop app is the *company* client.

1.2 Roles → capabilities (from `role.service.ts`)

Role	Gets	In the UI
ADMIN	All 18 permissions	Everything below.
WAREHOUSE	<code>VIEW_STOCK</code> , <code>SCAN_IN</code> , <code>SCAN_OUT</code> , <code>VIEW_TRUCK_STOCK</code> , <code>TRANSFER_STOCK_TO_TRUCK</code> , <code>VIEW_LOW_STOCK_ALERTS</code>	Inventory + scanning + stock trucks; no users/assets/ receipt-approval.
TECHNICIAN	<code>VIEW_ASSIGNED_TRUCK_STOCK</code> , <code>UPLOAD_RECEIPT</code> , <code>VIEW_LOW_STOCK_ALERTS</code>	"My Truck" + upload receipts only.

Principle: the desktop app must gate *navigation and every action button* on the `permissions` array from login — identical to the web app — never on a guessed role name.

2. How the web app does it (the reference IA)

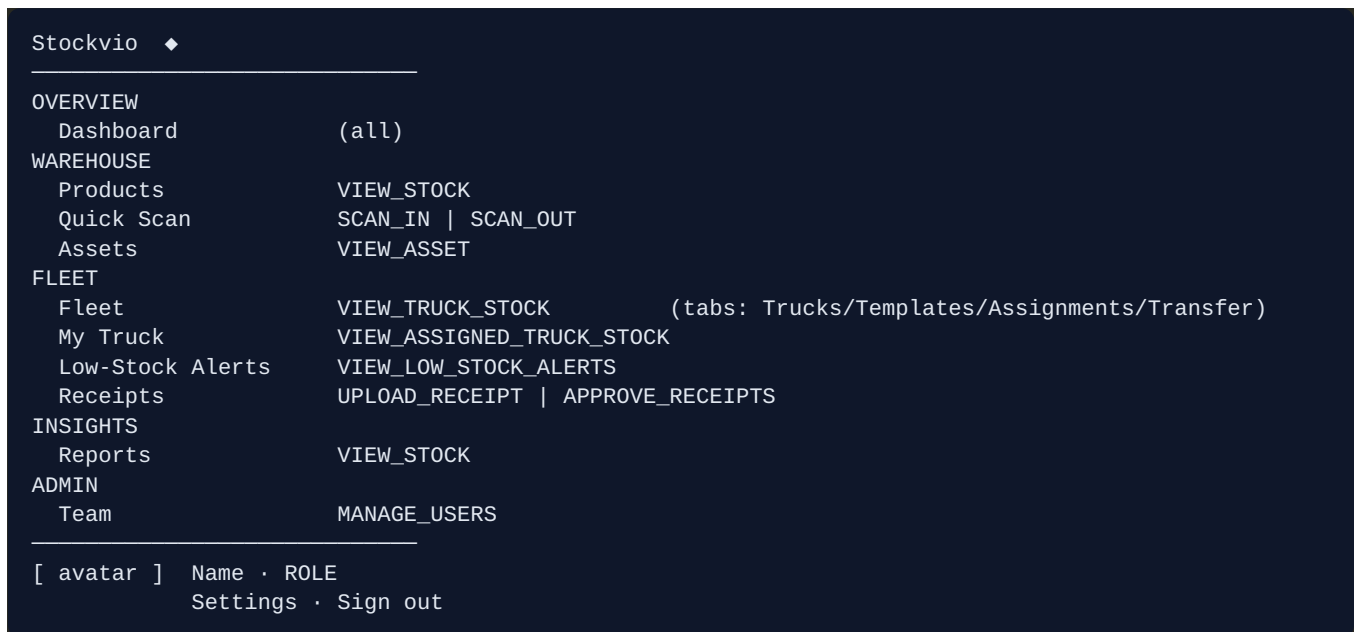
The web client (`inventory-system-api/web`) is the canonical pattern to mirror:

- **Left sidebar + top bar shell.** Sidebar items are filtered by permission; the active item uses a subtle light-indigo background, not a solid block.
- **Per-page anatomy:** `PageHeader` (title + description + one primary action) → optional filter/search row → a single `card` containing a table → modals for create/edit → explicit **loading / error / empty** states.
- **Fleet** is one page with **tabs** (Trucks · Templates · Assignments · Transfer).
- **Role gating** via `hasPermission(...)` ; buttons simply don't render when not allowed.
- **Design tokens:** indigo `#4F46E5` , slate ink scale, 8px inputs / 14px cards, soft borders `#E2E8F0` , readable light-indigo selection.

The Qt app already has equivalent page classes — this plan **re-architects their layout and IA to match**, rather than adding new pages.

3. Navigation / menu structure (role-aware)

Single left sidebar, grouped, filtered by permission. Mirrors the web exactly.



- **Top bar:** page title + breadcrumb, global search (`Ctrl/Cmd-K`), notifications, user menu.
- **Empty sidebar groups collapse** when a role has no items in them (e.g. a technician sees only Dashboard, My Truck, Low-Stock, Receipts, Settings).

4. Page-by-page layout

Each page: **Header** (title · subtitle · primary action) → **toolbar** (search/filters) → **content** (table/cards) → **states** (loading skeleton, empty, error) → **modals/drawer**.

4.1 Dashboard (all roles; `DashboardPage` / `TruckStockDashboardPage`)

- **Data:** `GET /reports/inventory-summary` , `/assets-summary` , `/products/low-stock` , `/truck-stock/low-stock` .
- **Layout:** 4 KPI **stat cards** (Total products, Qty on hand, Assets, Low-stock count) with icon chips and trend; a 2-column row — *Inventory summary* mini-chart + *Low-stock list* (top 7, click → Products filtered). Technician dashboard swaps to "My truck readiness" + "Items to restock."
- **Components:** StatCard, MiniBarChart, list card, "View all" links.

4.2 Products (`ItemsPage`)

- **Data:** `GET /products` (+ `/low-stock`), `POST/PUT/DELETE /products/:id` .
- **Header action:** Add product (`ADD_PRODUCT`).
- **Toolbar:** search (name/barcode/type/location), type & status filters, column-sort.
- **Table:** Product (name + model), Barcode (mono), Type, Location, **Qty** (amber when $\leq$ threshold), Status pill, row actions **Edit / Delete** (`ADD_PRODUCT`).
- **Detail drawer** (right slide-in) on row click: full fields + quantity history.
- **Modals:** Add/Edit product (shared form).
- **Pagination** footer.

4.3 Quick Scan (*ScanPage* ; *SCAN_IN* / *SCAN_OUT*)

- **Data:** `POST /products/scan-in` , `/scan-out` .
- **Layout:** large barcode field (auto-focus, Enter submits) + quantity stepper + big **In** (green) / **Out** (amber) buttons; live "recent scans" list; success/error toast. Designed for a handheld scanner workflow.

4.4 Assets (*AssetsPage*)

- **Data:** `GET /assets` , `POST/PUT/DELETE /assets/:id` .
- Same anatomy as Products: search, table (Asset, Type, Serial, Status pill, Added), **Add** (`ADD_ASSET`), row **Edit** (`EDIT_ASSET`) / **Delete** (`DELETE_ASSET`), detail drawer.

4.5 Fleet (*tabbed*; *TrucksPage* + *StockTemplatesPage* + *AssignmentsPage*)

One page, 4 tabs (gated): 1. **Trucks** (`MANAGE_TRUCK_STOCK`): table (Truck #, Plate, Technician, Status) + Add/Edit modal (technician dropdown from `GET /users` when `MANAGE_USERS`). 2. **Templates** (`MANAGE_TRUCK_STOCK`): card grid; create/edit modal with a **dynamic line-item editor** (productName, required, min, unit, category); delete with confirm; `GET /templates/:id` to load items for edit. 3. **Assignments** (`ASSIGN_TRUCK_STOCK`): table (Truck ↔ Template) + Assign modal (truck + template selects); un-assign. 4. **Transfer** (`TRANSFER_STOCK_TO_TRUCK`): form — warehouse product (with on-hand qty) → truck stock item → quantity → `POST /transfer-to-truck` .

4.6 My Truck (*MyTruckStockPage* ; *VIEW_ASSIGNED_TRUCK_STOCK*)

- **Data:** `GET /truck-stock/my-stock` ; `POST /use-item` ; `POST /receipts/upload` + `/receipts` .
- **Layout:** truck header card (number/plate/status) + stock table (Item, Category, On-truck vs Required with progress, Status); per-row **Use** modal (qty + notes); **Upload receipt** action. Friendly empty state when `404` (no truck assigned).

4.7 Low-Stock Alerts (*LowStockAlertsPage* ; *VIEW_LOW_STOCK_ALERTS*)

- **Data:** `GET /truck-stock/low-stock` (+ warehouse `/products/low-stock`).
- Grouped list: by truck / warehouse; each row shows current vs minimum, quick **Transfer/Restock** action where permitted.

4.8 Receipts (*ReceiptsPage* ; *APPROVE_RECEIPTS* *admin*, *UPLOAD_RECEIPT* *tech*)

- **Data:** `GET /receipts` ; `POST /:id/reconcile` ; `PATCH /:id/status` .
- **Admin view:** table (Date, Truck, Total, Status pill, File link) + actions **Reconcile** (shows matched/missing/extra/price-diff summary) / **Approve** / **Reject**.
- **Technician view:** their own uploads + status; **Upload** action.

4.9 Reports (*ReportsPage* ; *VIEW_STOCK* , *audit needs* *MANAGE_USERS*)

- **Data:** `/reports/inventory-summary` , `/assets-summary` , `/stock-movements` , `/audit-logs` .
- Tabs: **Summary** (KPI cards + charts), **Stock movements** (filterable table), **Audit log** (admin only). Each tab has **Export** (CSV/XLSX/PDF via `/exports/*`).

4.10 Team (*UsersPage* ; *MANAGE_USERS*)

- **Data:** `GET /users` ; invite/update/reset/delete; export users.

- Table (Name, Email, Role pill, status); **Invite** modal (name/email/role); row **Edit** (role), **Reset password** (reveals temp password banner), **Remove** (not self). Optional per-user permission toggles drawer (`assign/remove-permission`).

4.11 Settings (`SettingsPage` ; *all*)

- Profile (`PATCH /users/me`), **Change password** (`/auth/change-password`), Appearance (light/dark toggle — keep the theme switch), API connection info, Sign out. Force the change-password flow when `mustChangePassword` .

5. Component library (build once, reuse everywhere)

Component	Spec
Buttons	Primary (indigo), Secondary (white/outline), Ghost, Danger (red) — via a <code>variant</code> dynamic property so QSS styles them centrally.
Inputs	10px radius, 2px indigo focus ring; labelled; inline validation text.
Table	Sortable headers, hover row, light-indigo selection, sticky header, right-aligned numerics, row action cluster, pagination footer, skeleton rows while loading.
StatCard	Title, big value, icon chip (tinted), trend delta.
Status pill / Badge	good / warn / muted / brand tones (e.g. ACTIVE, LOW, PENDING, APPROVED).
Modal & Confirm dialog	Header + body + footer (Cancel / Primary); destructive actions always confirm.
Detail drawer	Right slide-in for record details (keeps context vs full page nav).
Toast	Transient success/error, bottom-center.
EmptyState / ErrorState / Loading	Consistent across every page.
SearchBar + filters	Debounced; chips for active filters.
Global search (Ctrl-K)	Already present (<code>GlobalSearchDialog</code>) — elevate to a command palette across pages/records.

6. Key user flows

1. **Onboarding (admin):** Login → Dashboard → Team → Invite user → (invitee gets temp password) → invitee logs in → **forced change-password** → Dashboard.
2. **Warehouse daily:** Login → Quick Scan (receive) → Products (verify) → Fleet ► Transfer (stock a truck) → Low-Stock Alerts (restock).
3. **Technician daily:** Login → My Truck → Use item (job) → Upload receipt → Low-Stock.
4. **Receipt approval (admin):** Receipts → open → Reconcile → review diff → Approve/Reject.
5. **Reporting:** Reports → Summary/Movements/Audit → Export.

7. UX improvements over today

- **Single source of truth for gating** — nav + actions from `permissions` (already wired on web).
- **Consistent page anatomy** (header/toolbar/content/states) so every screen feels the same.
- **Detail drawers** instead of dead-end dialogs for viewing records.
- **Loading skeletons & optimistic updates** for snappy feel; **toasts** for feedback.
- **Confirm dialogs** for all destructive actions; never delete on a single click.
- **Keyboard-first**: Ctrl-K search, Enter to submit scans, Esc to close modals, focus rings.
- **Status color system** shared with web (good/warn/danger/brand).
- **Empty states with a primary CTA** (e.g. "No products yet → Add product").
- **Responsive content area** (cards reflow; tables scroll horizontally with sticky header).
- **Accessibility**: visible focus, 4.5:1 contrast, large hit targets for scan workflows.

8. Mapping to existing Qt files (what changes)

Plan screen	Existing <code>.ui</code> / class	Action
Shell	<code>DashboardWindow</code> , <code>SidebarWidget</code> , <code>VerticalWidget</code>	Re-group nav, permission filter, subtle active state, Ctrl-K.
Dashboard	<code>DashboardPage</code> , <code>TruckStockDashboardPage</code>	Standardize KPI cards + 2-col summary; role variant.
Products / Assets	<code>ItemsPage</code> , <code>AssetsPage</code>	Unify toolbar + table + detail drawer; gate actions.
Quick Scan	<code>ScanPage</code>	Keep; enlarge targets; recent-scans list.
Fleet	<code>TrucksPage</code> , <code>StockTemplatesPage</code> , <code>AssignmentsPage</code>	Merge into one tabbed page (as on web).
My Truck	<code>MyTruckStockPage</code>	Truck header + progress table + Use/Upload.
Low-Stock	<code>LowStockAlertsPage</code>	Grouped alerts + quick actions.
Receipts	<code>ReceiptsPage</code>	Admin reconcile/approve; tech upload.
Reports	<code>ReportsPage</code>	Tabs + exports.
Team	<code>UsersPage</code>	Invite/edit/reset/remove + permissions drawer.
Settings	<code>SettingsPage</code>	Profile/password/appearance/connection.

9. Suggested implementation order

1. **Design system in QSS** (buttons via `variant` , inputs, table, cards, pills, drawer) — done once, every page benefits.
2. **Shell** (sidebar grouping + permission filter + top bar + Ctrl-K).
3. **Products + Assets** (the table pattern) → reuse for **Team**, **Receipts**, **Fleet/Trucks**.
4. **Dashboard KPIs** → **Fleet tabs** → **My Truck** → **Low-Stock** → **Reports**.

5. **Settings** + forced password change.

Because layout changes live in `.ui` files that can't be previewed in this environment, each screen should be built, run on Windows, and screenshot-checked before moving on — the plan above makes each screen a small, self-contained, verifiable unit.